Przetwarzanie języka naturalnego

Laboratorium nr 2

Od tekstu do tokenów: n-gramy, sprowadzanie form, odległość edycyjna i tokenizacja podsłowowa

Zasady

Czas na wykonanie: 2 tygodnie od ogłoszenia zadania.

Praca w parach, tak jak dotychczas.

Wszystkie narzędzia muszą być darmowe i otwarte. Żadnych płatnych API (OpenAI, Anthropic, Google) — wszystko liczymy lokalnie albo korzystamy z modeli otwartych na Hugging Face.

Zadanie na 3.0 nie wymaga sprawozdania ani odpowiedzi ustnej. Zadania na 4.0 i 5.0 wymagają obu.

Zadanie na 4.0 obejmuje wszystko z 3.0, a zadanie na 5.0 — wszystko z 4.0.

Cel laboratorium

Przechodzimy pełną drogę od surowego pliku tekstowego do jednostek, na których pracuje model językowy: czyszczenie tekstu, podział na zdania i wyrazy, n-gramy, sprowadzanie form do rdzenia i lematu, mierzenie podobieństwa napisów, a na końcu — tokenizacja podsłowowa, czyli to, czego naprawdę używają dzisiejsze modele.

Materiał teoretyczny znajduje się w Wykładzie 3 „Podstawy przetwarzania języka naturalnego”. Do wykładu dołączony jest komplet działających przykładów (pliki example_01.py … example_18.py) — warto z nich korzystać jako punktu wyjścia.

Środowisko

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

pip install nltk stop-words spacy matplotlib textdistance
pip install transformers tokenizers sentencepiece
python -m spacy download pl_core_news_sm
```

Przy pierwszym uruchomieniu nltk pobierze potrzebne dane (nltk.download). Biblioteki transformers i tokenizers pobierają wyłącznie pliki tokenizatorów — kilkaset kilobajtów, nie wagi modeli.

Teksty do analizy

Tekst polski — dowolna pozycja z Wolnych Lektur w formacie .txt, na przykład:

https://wolnelektury.pl/media/book/txt/pan-tadeusz.txt

Tekst angielski — dowolna pozycja z Project Gutenberg, na przykład Hamlet:

https://www.gutenberg.org/ebooks/27761

Możesz wybrać inne teksty — warunek jest jeden: mają być w domenie publicznej i mieć co najmniej kilkadziesiąt tysięcy znaków.

Zadanie na 3.0

Bez sprawozdania i bez odpowiedzi ustnej. Wystarczy działający kod i wyniki.

Część A — n-gramy w tekście polskim

1. Pobierz wybraną książkę w formacie .txt i wczytaj ją w Pythonie.
2. Oczyść tekst: usuń znaki specjalne, zamień litery na małe, zwiń wielokrotne białe znaki.
3. Wykonaj tokenizację na zdania i na wyrazy. Podaj, ile zdań i ile wyrazów ma tekst.
4. Wygeneruj unigramy, bigramy i trigramy słowne.
5. Policz częstość występowania n-gramów (collections.Counter albo nltk).
6. Narysuj histogramy dziesięciu najczęstszych bigramów i dziesięciu najczęstszych trigramów (matplotlib).
7. Odpowiedz w komentarzu w kodzie lub w pliku tekstowym na pytania:
   - Jakie bigramy dominują w analizowanym tekście?
   - Czy trigramy pokazują powtarzalne zwroty charakterystyczne dla autora?
   - Jak różni się rozkład unigramów od rozkładu bigramów?

Punkt wyjścia: example_16.py i example_17.py z materiałów do wykładu.

Część B — sprowadzanie form w tekście angielskim

1. Pobierz tekst z literatury angielskiej (np. Hamlet z Project Gutenberg).
2. Wybierz z niego 5 zdań.
3. Dla tych zdań wykonaj tokenizację, stemming (jeden algorytm) i lematyzację (jeden algorytm).
4. Zestaw wyniki w tabeli: token → rdzeń → lemat. Wskaż co najmniej trzy słowa, dla których stemming i lematyzacja dały różne wyniki.

Punkt wyjścia: example_08.py i example_09.py.

Część C — odległość edycyjna

1. Dysponujesz zdaniami: „Ala ma kota” oraz „Ala posiada kota”.
2. Na podstawie biblioteki textdistance zbadaj odległość Levenshteina między nimi.
3. Wyjaśnij uzyskany wynik — z ilu i z jakich operacji edycyjnych się składa.

Punkt wyjścia: example_18.py.

Co oddajesz

Na platformie Delta wgraj archiwum o nazwie:

```
LAB2_Nazwisko_Imie_Nazwisko_Imie.zip
```

Archiwum powinno zawierać wybrane pliki .txt z tekstami oraz kody źródłowe w języku Python. Zadanie na 3.0 nie wymaga sprawozdania ani odpowiedzi ustnej.

Zadanie na 4.0

Wszystko z zadania na 3.0, a ponadto punkty poniżej. Wymagane sprawozdanie w formacie PDF oraz odpowiedź ustna na najbliższych laboratoriach.

Część D — porównanie stemmerów

1. Dla tekstu angielskiego porównaj dwa różne stemmery (np. Porter i Lancaster albo Porter i Snowball).
2. Znajdź co najmniej pięć słów, dla których algorytmy dają różne rdzenie. Zastanów się, który wynik jest sensowniejszy i dlaczego.
3. Wskaż przykład nadmiernego obcinania (dwa różne wyrazy dostają ten sam rdzeń) i przykład niedostatecznego (dwie formy tego samego wyrazu nie schodzą się).

Część E — polszczyzna wymaga innych narzędzi

1. Wykonaj tokenizację, stemming i lematyzację dla fragmentu tekstu polskiego, korzystając z rozwiązań dedykowanych dla tego języka (spaCy z modelem pl_core_news_sm, Stanza albo Morfeusz2).
2. Sprawdź, co się stanie, gdy do polskich słów zastosujesz angielski stemmer (SnowballStemmer("english")). Pokaż wyniki dla co najmniej pięciu wyrazów.
3. Wypisz listę języków obsługiwanych przez SnowballStemmer i skomentuj obecność lub brak polskiego.

Punkt wyjścia: example_08.py i example_09.py.

Część F — więcej miar podobieństwa

1. Wybierz jeszcze trzy miary z biblioteki textdistance (poza odległością Levenshteina).
2. Opisz zasadę działania każdej z nich.
3. Porównaj wyniki wszystkich czterech miar dla pary „Ala ma kota” / „Ala posiada kota” oraz dla co najmniej trzech innych par, które sam dobierzesz (np. z literówkami, z przestawionymi literami, z różnicą tylko w diakrytykach).
4. Wskaż parę, dla której miary najbardziej się różnią, i wyjaśnij dlaczego.

Co oddajesz

Archiwum LAB2_Nazwisko_Imie_Nazwisko_Imie.zip zawierające pliki .txt, kody źródłowe oraz sprawozdanie w formacie PDF. Sprawozdanie powinno zawierać:

- analizę uzyskanych histogramów,
- odpowiedzi na pytania z części A,
- definicje wskazujące na różnice między stemmingiem a lematyzacją oraz krótkie podsumowanie wykorzystanych metod,
- porównanie wyników stemmingu i lematyzacji dla obu języków,
- zasady działania algorytmów odległości edycyjnej i porównanie wyników osiągniętych przez poszczególne miary.

Prześlij archiwum w terminie i podejdź do odpowiedzi ustnej na najbliższych laboratoriach.

Zadanie na 5.0

Wszystko z zadania na 4.0, a ponadto punkty poniżej.

Część G — stop words i nazwy własne

1. Usuń stop words z tekstu polskiego (biblioteka stop-words; nltk nie zawiera listy dla polszczyzny).
2. Powtórz analizę n-gramów z części A na tekście po usunięciu stop words. Narysuj te same histogramy.

3. Porównaj oba warianty — z usuniętymi stop words i bez. Czy różnice są istotne? Który wariant lepiej pokazuje treść tekstu, a który jego styl?
4. W tekście z części A odnajdź nazwy własne. W języku polskim jest to o tyle łatwiejsze, że zaczynają się od wielkiej litery — zwróć jednak uwagę, żeby słowo nie stało na początku zdania (wielka litera w środku zdania to znacznie większa szansa, że mamy do czynienia z nazwą własną). Zadanie zrealizuj z wykorzystaniem wyrażeń regularnych.
5. Wskaż dziesięć najczęściej występujących nazw własnych.

Część H — własny tokenizator podsłowowy

Ta część jest bezpośrednim rozwinięciem wykładu o BPE. Korzystamy z biblioteki tokenizers od Hugging Face — jest darmowa i działa lokalnie.

1. Wytrenuj własny tokenizator BPE na wybranym polskim korpusie (może to być ten sam tekst, co w części A, albo większy zbiór z Wolnych Lektur).
2. Powtórz uczenie dla trzech różnych rozmiarów słownika, na przykład 1 000, 5 000 i 30 000 tokenów.
3. Dla każdego z trzech tokenizatorów sprawdź podział słów: „nieprzetłumaczalny", „hiperparametryzacja", „kotkowi" oraz jednego wyrazu wybranego przez siebie. Zestaw wyniki w tabeli.
4. Wybierz pięć zdań po polsku i pięć po angielsku. Policz, ile tokenów daje każde zdanie dla:
   - Twojego tokenizatora (wariant 5 000),
   - tokenizatora gpt2,
   - tokenizatora allegro/herbert-base-cased,
   - tokenizatora modelu Bielik (speakleash/Bielik-1.5B-v3) — pobierany jest wyłącznie plik tokenizatora.
5. Zestaw wyniki w tabeli i narysuj wykres słupkowy: średnia liczba tokenów na zdanie, osobno dla polskiego i angielskiego.
6. Odpowiedz na pytania:
   - Który tokenizator jest najbardziej ekonomiczny dla tekstu polskiego, a który dla angielskiego?
   - Jak zmienia się liczba tokenów wraz ze wzrostem rozmiaru słownika Twojego tokenizatora? Czy zależność jest liniowa?
   - Dlaczego tokenizator wytrenowany na jednej książce wypada dobrze na tej książce, a gorzej na dowolnym innym tekście?
   - Jaki jest związek między tym ćwiczeniem a pojęciem „podatku językowego" z wykładu?

Punkt wyjścia: example_14.py (uczenie własnego tokenizatora) i example_15.py (porównanie z gotowymi).

Część I — porównanie wszystkich metod

1. Zestaw w jednej tabeli wyniki dla tego samego fragmentu tekstu polskiego, przetworzonego czterema sposobami: same wyrazy, wyrazy po lematyzacji, wyrazy bez stop words oraz tokeny podsłowowe z Twojego tokenizatora.
2. Dla każdego wariantu podaj liczbę jednostek i rozmiar uzyskanego słownika (liczbę unikalnych jednostek).
3. Skomentuj, który wariant najmocniej zmniejsza słownik i jakim kosztem.
4.

Co oddajesz

Archiwum LAB2_Nazwisko_Imie_Nazwisko_Imie.zip zawierające pliki .txt, kody źródłowe, plik wytrenowanego tokenizatora (tokenizer_pl.json) oraz sprawozdanie w formacie PDF. Sprawozdanie powinno zawierać wszystko z poziomu 4.0, a ponadto:

- porównanie wyników z usunięciem stop words i bez ich usuwania,
- omówienie wyrażenia regularnego użytego do wyszukiwania nazw własnych i wyników, jakie dało,
- tabelę i wykres z porównania tokenizatorów wraz z komentarzem,
- odpowiedzi na pytania z części H,
- tabelę porównawczą z części I wraz z wnioskiem.

Prześlij archiwum w terminie i podejdź do odpowiedzi ustnej na najbliższych laboratoriach.